CYBERSECURITY HOME LAB

Linux Access Control

TCP Wrappers

Host-Based Access Control for Network Services

Practical Lab — vsftpd · tcpdmatch · /etc/hosts.allow · /etc/hosts.deny

Parrot Security OS	**Ubuntu 26.04 VM**
SERVER — vsftpd + TCP Wrappers	*CLIENT — ftp connection tester*

[Ihar Shcharbitski] · [16.05.2026]

1. Why Are TCP Wrappers Important?

TCP Wrappers is a host-based access control system. Host-based means it runs directly on the machine being protected — in this lab, that is the Parrot OS machine acting as a server.

Think of it this way: vsftpd is a room inside the Parrot machine. TCP Wrappers is the security guard at the entrance. Before any client — such as the Ubuntu VM — can walk in, the guard checks a rulebook. If the client's IP address is on the allowed list, the door opens. If it is on the denied list, the door slams shut. Parrot OS controls that rulebook entirely.

File	Purpose
`/etc/hosts.allow`	The allow list. Always checked first. A match here grants access immediately — the deny file is never consulted.
`/etc/hosts.deny`	The deny list. Only checked if no match was found in hosts.allow. A match here blocks the connection.

> **Defence in Depth:** TCP Wrappers operates at the application layer, independently of the network firewall. Even if a firewall rule is misconfigured and a packet slips through, TCP Wrappers on the server machine provides a second independent layer of enforcement.

> **Evaluation Order:** hosts.allow is always read first. A match there grants access and hosts.deny is never consulted. If no match is found in hosts.allow, hosts.deny is checked. If no match is found in either file, access is granted by default — which is why a deny-all rule in hosts.deny is essential.

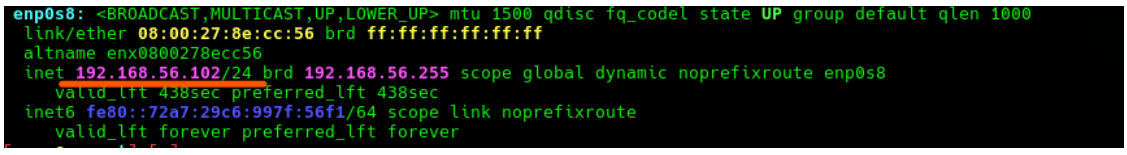
2. Preparing the Lab Environment

This lab uses two virtual machines. Parrot OS serves as the protected server — it runs vsftpd and enforces all TCP Wrappers rules. Ubuntu 26.04 serves as the client — its only role is to connect to Parrot's FTP server to prove the rules work.

2.1 Finding IP Addresses

Before writing any rules, the IP address of each machine must be identified. These addresses will be referenced throughout the lab.

Parrot OS — Server IP



Parrot OS IP address (server machine)

Ubuntu VM — Client IP

```
enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
 link/ether 08:00:27:55:73:d0 brd ff:ff:ff:ff:ff:ff
 altname enx0800275573d0
 inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s8
    valid_lft 516sec preferred_lft 516sec
 inet6 fe80::dc5b:36a:ca45:65f0/64 scope link noprefixroute
    valid_lft forever preferred_lft forever
```

Ubuntu VM IP address (client machine)

> The IP address in /etc/hosts.allow will be the Ubuntu VM's IP — because Ubuntu is the client requesting access to the server.

2.2 Verifying Network Connectivity

Before proceeding, both machines must be confirmed to be reachable from one another. This is done using the ping command from Parrot OS to the Ubuntu VM.

```
ping -c 4 <UBUNTU_VM_IP>
```

```
┌─[user@parrot]─[~]
└──╼ $ping -c 4 192.168.56.101
PING 192.168.56.101 (192.168.56.101) 56(84) bytes of data.
64 bytes from 192.168.56.101: icmp_seq=1 ttl=64 time=0.568 ms
64 bytes from 192.168.56.101: icmp_seq=2 ttl=64 time=0.562 ms
64 bytes from 192.168.56.101: icmp_seq=3 ttl=64 time=0.558 ms
64 bytes from 192.168.56.101: icmp_seq=4 ttl=64 time=0.543 ms

--- 192.168.56.101 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3055ms
rtt min/avg/max/mdev = 0.543/0.557/0.568/0.009 ms
```

Successful ping from Parrot OS to Ubuntu VM (0% packet loss)

> If ping fails at this stage, check that both VMs are assigned to the same Host-Only network adapter in VirtualBox settings. Do not proceed until connectivity is confirmed.

2.3 Installing and Starting vsftpd on Parrot OS

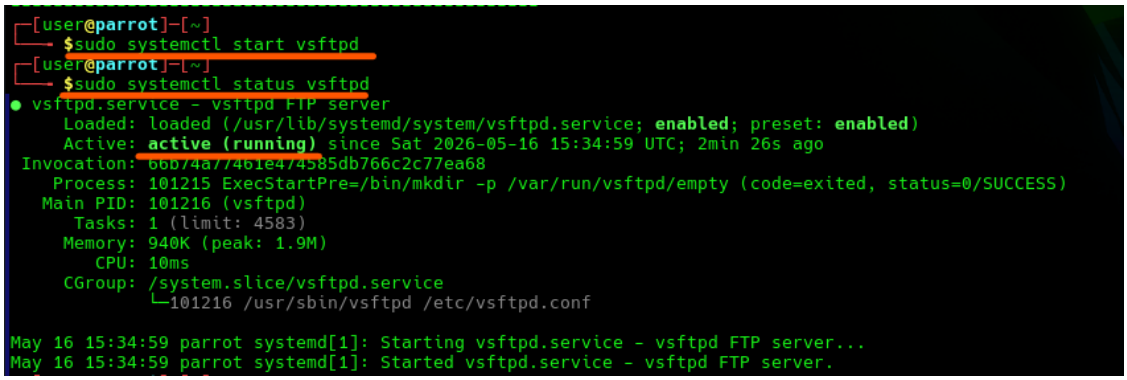
vsftpd (Very Secure FTP Daemon) is the FTP server that TCP Wrappers will protect. It is installed and started on Parrot OS.

```
sudo apt update && sudo apt install vsftpd -y
```

```
┌─[user@parrot]─[~]
└──╼ $sudo apt install vsftpd -y
The following package was automatically installed and is no longer required:
  libwoff1
Use 'sudo apt autoremove' to remove it.

Installing:
  vsftpd

Summary:
  Upgrading: 0, Installing: 1, Removing: 0, Not Upgrading: 0
  Download size: 144 kB
  Space needed: 352 kB / 50.3 GB available

Get:1 https://deb.parrot.sh/parrot echo/main amd64 vsftpd amd64 3.0.5-0.2 [144 kB]
Fetched 144 kB in 0s (341 kB/s)
Preconfiguring packages ...
Selecting previously unselected package vsftpd.
(Reading database ... 568023 files and directories currently installed.)
Preparing to unpack .../vsftpd_3.0.5-0.2_amd64.deb ...
Unpacking vsftpd (3.0.5-0.2) ...
Setting up vsftpd (3.0.5-0.2) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/vsftpd.service' → '/usr/lib/systemd/system/vsftpd.service'.
/usr/lib/tmpfiles.d/vsftpd.conf:1: Line references path below legacy directory /var/run/, updating /var/run/vsftpd/empty → /run/vsftpd/empty; please update the tmpfiles.d/ drop-in file accordingly.
Processing triggers for man-db (2.13.1-1) ...
-----------------------------------------------
[!] Scanning application launchers
Removing duplicate or broken launchers...
[!] Launchers have been successfully updated!
```

Fig. 4 — vsftpd installation on Parrot OS

```
sudo systemctl start vsftpd && sudo systemctl enable vsftpd
```

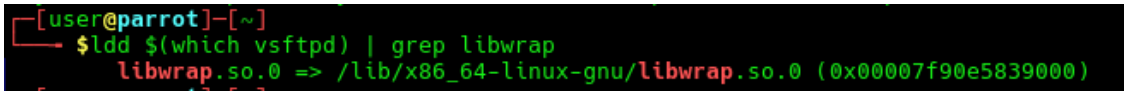


vsftpd started and enabled — 'active (running)' confirms success

2.4 Verifying TCP Wrappers (libwrap) Support in vsftpd

Not all vsftpd builds include TCP Wrappers support. This check confirms that the installed binary is linked against libwrap and will therefore obey the rules in hosts.allow and hosts.deny.

```
ldd $(which vsftpd) | grep libwrap
```



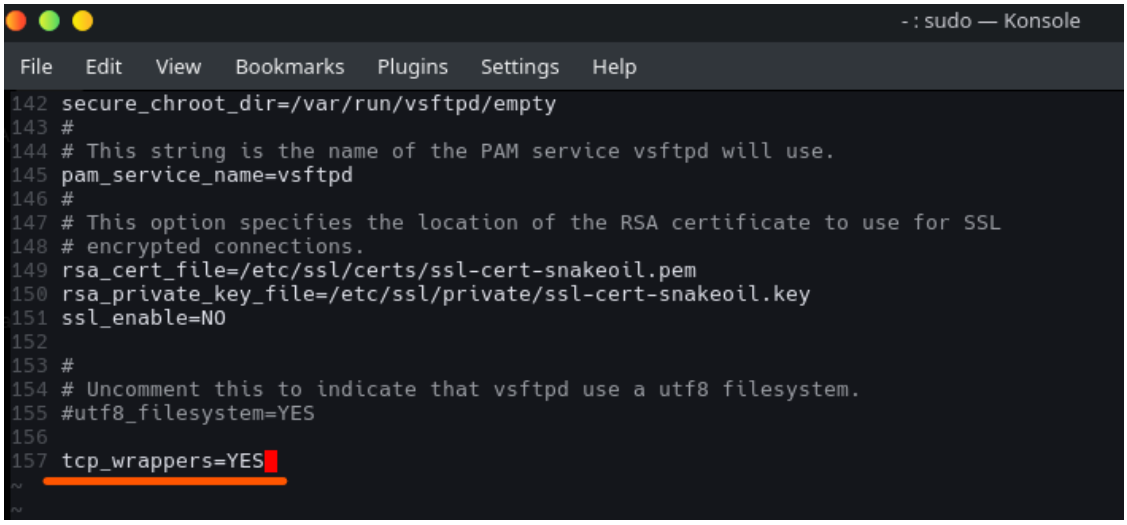
libwrap present in vsftpd binary — TCP Wrappers support confirmed

Part of the command	What it does
`which vsftpd`	Finds the full filesystem path of the vsftpd binary.
`ldd`	Lists all shared libraries the binary depends on at runtime.
`grep libwrap`	Filters the output to show only the TCP Wrappers library line. If this line appears, vsftpd will honour the rule files.

libwrap being present in the binary output is necessary but not sufficient. The vsftpd configuration file must also explicitly enable TCP Wrappers support, as shown in the next step.

2.5 Enabling TCP Wrappers in the vsftpd Configuration File

The vsftpd configuration file at `/etc/vsftpd.conf` must contain the directive `tcp_wrappers=YES` for vsftpd to consult the rule files on each incoming connection. Without this line, vsftpd ignores hosts.allow and hosts.deny entirely — even if libwrap is present.



tcp_wrappers=YES added to /etc/vsftpd.conf

> To exit Vim after editing: press [Esc] to leave INSERT mode, then type :wq and press [Enter] to save and quit.

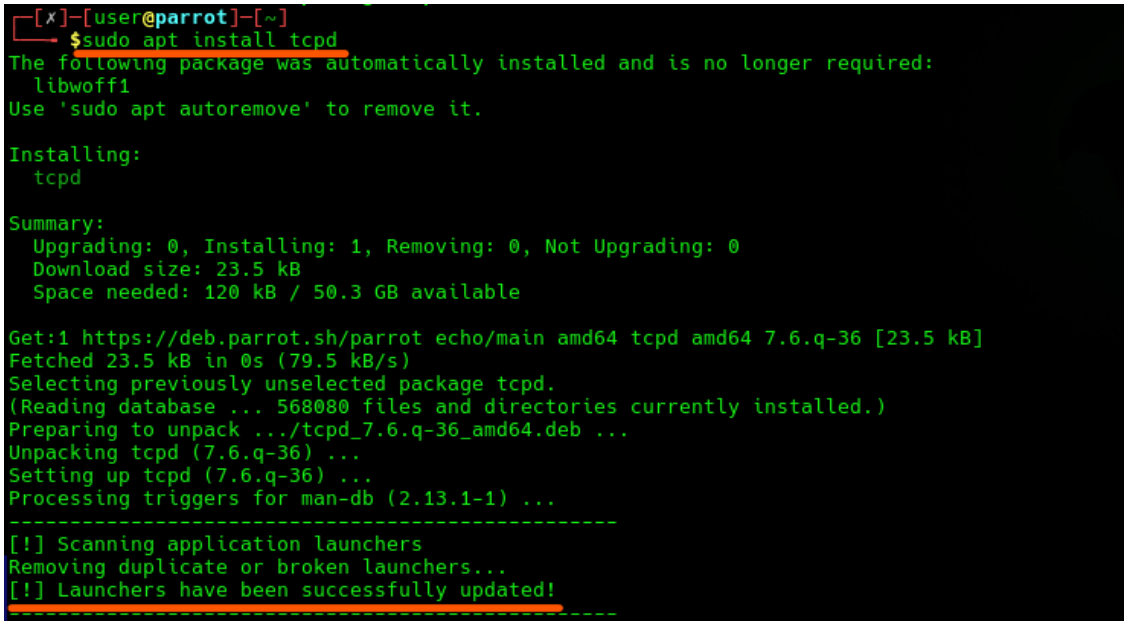
After saving the configuration file, restart vsftpd to apply the change:

```
sudo systemctl restart vsftpd
```

2.6 Installing tcpdmatch on Parrot OS

tcpdmatch is a simulation utility included in the tcp_wrappers package. It reads hosts.allow and hosts.deny and predicts the allow/deny decision for a given service and client IP — without making a real connection. This makes it safe to test rules before they affect live traffic.

```
sudo apt install tcpd -y
```



tcpdmatch installed on Parrot OS

2.7 Installing the FTP Client on Ubuntu VM

The Ubuntu VM requires only the ftp client package. This is the tool used to make real connection attempts to Parrot's FTP server, proving that the rules work in practice and not only in simulation.

```
sudo apt install ftp -y
```

```
shcherba@Ubuntu-26-04:~$ sudo apt install ftp
[sudo: authenticate] Password:
ftp is already the newest version (20260211-1).
ftp set to manually installed.
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2
```

ftp client installed on Ubuntu VM

3. Baseline Verification

A baseline test is performed before any rules are applied. This establishes a known-good state: both rule files are empty and the FTP server is reachable. If something breaks after a configuration change, the baseline confirms that the issue was introduced by that change — not a pre-existing problem.

3.1 Confirming Both Rule Files Are Empty

The following commands print the current contents of `/etc/hosts.allow` and `/etc/hosts.deny` on Parrot OS. On a fresh installation, both files contain only comment lines (lines beginning with #). Comments are ignored by TCP Wrappers — no active rules exist yet.

```
┌─[user@parrot]─[~]
└──╼ $cat /etc/hosts.allow
# /etc/hosts.allow: list of hosts that are allowed to access the system.
#                   See the manual pages hosts_access(5) and hosts_options(5).
#
# Example:    ALL: LOCAL @some_netgroup
#             ALL: .foobar.edu EXCEPT terminalserver.foobar.edu
#
# If you're going to protect the portmapper use the name "rpcbind" for the
# daemon name. See rpcbind(8) and rpc.mountd(8) for further information.
#
```

/etc/hosts.allow — comments only, no active rules

```
┌─[user@parrot]─[~]
└──╼ $cat /etc/hosts.deny
# /etc/hosts.deny: list of hosts that are _not_ allowed to access the system.
#                  See the manual pages hosts_access(5) and hosts_options(5).
#
# Example:    ALL: some.host.name, .some.domain
#             ALL EXCEPT in.fingerd: other.host.name, .other.domain
#
# If you're going to protect the portmapper use the name "rpcbind" for the
# daemon name. See rpcbind(8) and rpc.mountd(8) for further information.
#
# The PARANOID wildcard matches any host whose name does not match its
# address.
#
# You may wish to enable this to ensure any programs that don't
# validate looked up hostnames still leave understandable logs. In past
# versions of Debian this has been the default.
# ALL: PARANOID

┌─[user@parrot]─[~]
```

/etc/hosts.deny — comments only, no active rules

> With no active rules in either file, TCP Wrappers applies its default behaviour: all connections are permitted. This is the permissive default that the deny-all rule in the next section will override.

3.2 Baseline FTP Connection from Ubuntu

With no restrictions active, the Ubuntu VM connects to Parrot's FTP server successfully. This confirms the server is reachable and functioning correctly before any access control rules are applied.

```
ftp <PARROT_OS_IP>
```

```
shcherba@Ubuntu-26-04:~$ ftp 192.168.56.102
Connected to 192.168.56.102.
220 (vsFTPd 3.0.5)
Name (192.168.56.102:shcherba):
```

Baseline FTP connection from Ubuntu to Parrot OS — successful

The 220 (vsFTPd) welcome banner confirms the connection was accepted. This screenshot serves as the before reference for all subsequent tests.

4. Configuring the Deny-All Rule

All configuration in this section is performed on Parrot OS. The approach follows professional best practice: deny all access by default, then explicitly permit only what is required.

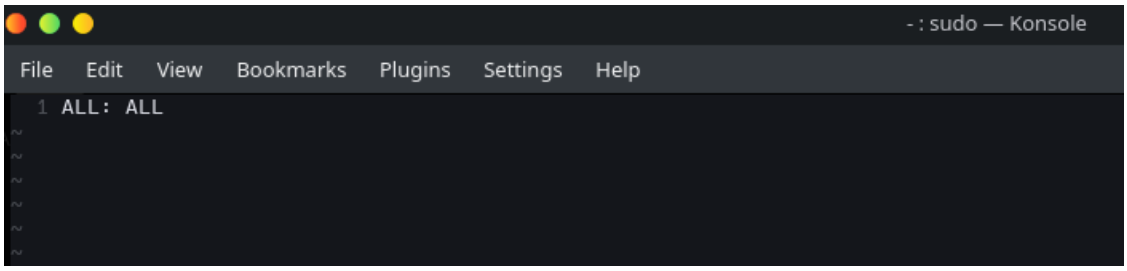
> **Principle of Least Privilege:** The deny-all rule directly implements Least Privilege: no client receives access unless it has been explicitly granted. This is far more secure than the default allow-all behaviour, which permits any client that has not been explicitly denied.

4.1 Applying the Deny-All Rule

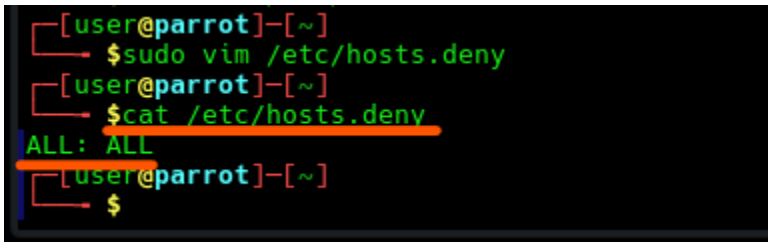
The following rule is added to `/etc/hosts.deny` on Parrot OS:

```
ALL : ALL
```

Token	Meaning
`ALL` (first position)	Applies to every service that consults TCP Wrappers — vsftpd, SSH, and any other wrapped service.
`:`	Separator between service list and client list.
`ALL` (second position)	Matches every client — any IP address, any hostname, from anywhere.



ALL : ALL deny-all rule written in /etc/hosts.deny



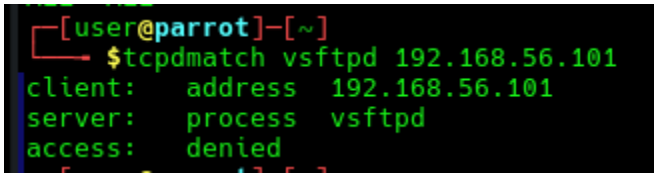
cat /etc/hosts.deny — rule confirmed saved

> No service restart is required. TCP Wrappers reads both rule files on every new incoming connection. The deny-all rule becomes active the moment the file is saved.

4.2 Testing the Rule with tcpdmatch

Before verifying with a real connection, tcpdmatch is used to simulate the outcome. This is important because it also shows exactly which file and which line number triggered the decision — useful diagnostic information when debugging complex rule sets.

```
tcpdmatch vsftpd <UBUNTU_VM_IP>
```



tcpdmatch output: 'access: denied' for Ubuntu VM IP

Output field	Meaning
`client: address ...`	The IP address tcpdmatch is simulating as the connecting client.
`server: process vsftpd`	The service being evaluated.
`matched: /etc/hosts.deny line N`	The specific file and line number that produced this verdict.
`access: denied`	The final decision TCP Wrappers would apply to a real connection from this client.

> tcpdmatch evaluates only the contents of hosts.allow and hosts.deny. It does not verify whether vsftpd has tcp_wrappers=YES in its configuration file. A real connection test (shown below) is therefore always necessary to confirm end-to-end enforcement.

4.3 Confirming Enforcement with a Real Connection

With the deny-all rule active, the Ubuntu VM attempts to connect to Parrot's FTP server. The connection should now be refused.

```
ftp <PARROT_OS_IP>
```

```
shcherba@Ubuntu-26-04:~$ ftp 192.168.56.102
Connected to 192.168.56.102.
421 Service not available.
```

FTP connection refused from Ubuntu — deny-all rule is active

The absence of the 220 (vsFTPd) welcome banner — which was present in the baseline test — confirms that TCP Wrappers is intercepting and blocking the connection before vsftpd responds.

> **Defence in Depth:** This demonstrates a second independent enforcement layer. Even if a network firewall rule were misconfigured and the packet reached port 21, TCP Wrappers on Parrot OS would still block it at the application layer.

5. Configuring the Allow Rule for a Specific Client

With the deny-all rule in place, access is now explicitly granted to the Ubuntu VM. The allow rule is added to /etc/hosts.allow on Parrot OS. Because hosts.allow is evaluated first, a match here grants access immediately — the deny file is never consulted.

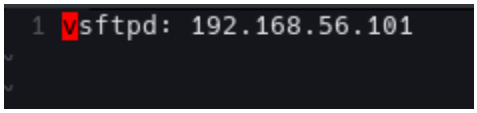
5.1 Adding the Allow Rule

The following rule is added to `/etc/hosts.allow` on Parrot OS, replacing <UBUNTU_VM_IP> with the actual Ubuntu IP address:

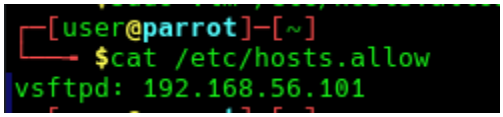
```
vsftpd : <UBUNTU_VM_IP>
```

Token	Meaning
`vsftpd`	This rule applies to the vsftpd service only — not SSH or any other service. Targeting a specific service is safer than using ALL.

:	Separator.
<UBUNTU_VM_IP>	The IP address of the Ubuntu VM — the only client permitted to reach vsftpd on this machine.



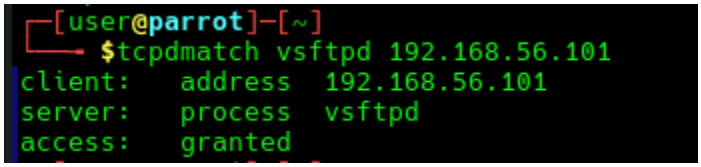
vsftpd allow rule written in /etc/hosts.allow for Ubuntu VM



cat /etc/hosts.allow — allow rule confirmed saved

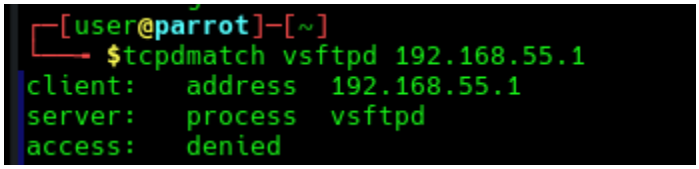
5.2 Verifying with tcpdmatch — Allow and Deny Both Confirmed

tcpdmatch is run twice: once with the Ubuntu IP (expected result: granted) and once with an arbitrary unknown IP (expected result: denied). This side-by-side comparison confirms both rules are functioning correctly.



tcpdmatch: Ubuntu IP granted via hosts.allow / unknown IP denied via hosts.deny

The output for the Ubuntu IP shows 'matched: /etc/hosts.allow' — confirming hosts.allow was evaluated first and the allow rule took effect. The unknown IP falls through to hosts.deny and is blocked by the deny-all rule.



tcpdmatch: arbitrary IP denied — deny-all rule still in effect

> **Allow-listing:** Only explicitly permitted clients can connect. All others — including unknown or future threats — are blocked automatically. This is significantly stronger than block-listing, which only stops known-bad addresses and permits everything else by default.

5.3 Live Verification — Ubuntu Connects Successfully

The allow rule is now validated with a real FTP connection from the Ubuntu VM to Parrot OS.

```
ftp <PARROT_OS_IP>
```

```
shcherba@Ubuntu-26-04:~$ ftp 192.168.56.102
Connected to 192.168.56.102.
220 (vsFTPd 3.0.5)
Name (192.168.56.102:shcherba):
```

FTP connection from Ubuntu to Parrot OS — allowed by hosts.allow rule

The 220 (vsFTPd) banner is visible again, confirming the connection was accepted. The full cycle — baseline, deny-all, allow-specific — has been demonstrated with real network traffic.

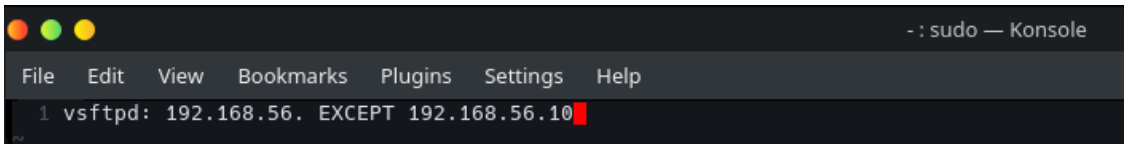
6. Advanced Rule: Subnet Allow with Targeted Exception

This section extends the configuration beyond a single IP. Rather than allowing one specific host, the rule permits any machine on the entire internal subnet — while explicitly excluding one designated host. This simulates a realistic scenario in which an entire internal network is trusted except for one machine that has been flagged as untrustworthy.

6.1 Applying the Subnet Rule with EXCEPT

The allow rule in `/etc/hosts.allow` is updated to the following. The trailing dot after the third octet is TCP Wrappers' subnet shorthand — it matches all addresses in the 192.168.56.0/24 range.

```
vsftpd : 192.168.56. EXCEPT <EXCLUDED_HOST_IP>
```

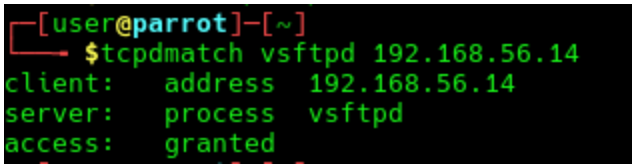


Subnet allow rule with EXCEPT clause written in /etc/hosts.allow

Token	Meaning
`192.168.56.`	Subnet shorthand — matches any IP address from 192.168.56.0 to 192.168.56.255 (the entire /24 subnet, consistent with the 255.255.255.0 subnet mask).
`EXCEPT`	Exclusion operator — the address or pattern that follows is removed from the matched set.
`<EXCLUDED_HOST_IP>`	The specific host that is denied access despite being in the otherwise trusted subnet.

6.2 Testing a Trusted Host in the Subnet

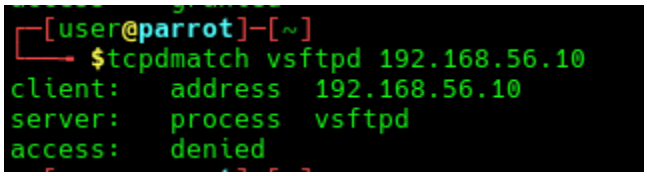
tcpdmatch is used to simulate a connection from a host that is within the allowed subnet and not the excluded address. The result should be granted.



tcpdmatch: host within 192.168.56.0/24 — access granted

6.3 Testing the Excluded Host

tcpdmatch is run again with the excluded IP address. Despite being in the subnet, this host should be denied.



tcpdmatch: excluded host — access denied via EXCEPT clause

The EXCEPT clause confirms targeted, granular access control: an entire subnet can be trusted while individual addresses within it remain blocked. This removes the need for a separate deny rule for the excluded host.

7. Summary

This lab demonstrated the configuration and verification of TCP Wrappers as a host-based access control mechanism on Linux. The following sequence was completed and validated with both simulated (tcpdmatch) and live (ftp client) tests:

1. A deny-all policy was implemented by adding ALL : ALL to /etc/hosts.deny, restricting all incoming connections to wrapped services by default.
2. An explicit allow rule was configured in /etc/hosts.allow to permit access from a trusted client — the Ubuntu VM — while all other hosts remained blocked.
3. The configuration was extended to allow connections from an entire subnet while excluding one specific host using the EXCEPT keyword.

TCP Wrappers remains relevant in modern Linux environments because it provides an additional, independent layer of network-level access control for services. By using /etc/hosts.allow and /etc/hosts.deny, administrators can restrict which hosts or subnets may interact with specific services before a connection is fully established at the application level.

Although TCP Wrappers is considered a legacy technology — with many modern environments relying on iptables, nftables, or cloud security groups for perimeter control — the underlying security principles it demonstrates remain foundational:

Security Principle	How It Was Demonstrated in This Lab
Principle of Least Privilege	ALL : ALL in hosts.deny ensures no access exists by default. Every permitted connection must be explicitly declared.
Defence in Depth	TCP Wrappers enforces access at the application layer, independently of any network firewall. Two distinct enforcement points protect the same service.
Allow-listing	Only known-good IP addresses are permitted. Unknown or future threats are blocked automatically without requiring additional rules.
Granular Access Control	The EXCEPT keyword allows an entire subnet to be trusted while individual addresses within it are selectively denied.

The End.